



Stack and Queue

CSE 2104: Data Structure and Algorithms-I Lab

Stack

- A linear data structure that follows the Last In, First Out (LIFO) principle.
- The last element added to the stack is the first one to be removed.
- Think of it like a stack of plates: you can only add or remove the top plate.

Stack Operations

- `stack.push()`
- `stack.pop()`
- `stack.top()`
- `stack.empty()`
- `stack.size()`

Stack Operations

stack.push()

stack.pop()

stack.top()

stack.empty()

stack.size()

stack.push(5);



5

Stack Operations

stack.push()

stack.pop()

stack.top()

stack.empty()

stack.size()

stack.push(5);

stack.push(4);

4
5

Stack Operations

stack.push()

stack.pop()

stack.top()

stack.empty()

stack.size()

stack.push(5);

stack.push(4);

stack.push(3);

3
4
5

Stack Operations

`stack.push()`

`stack.pop()`

`stack.top()`

`stack.empty()`

`stack.size()`

`stack.push(5);`

`stack.push(4);`

`stack.push(3);`

`stack.pop();`

4
5

Stack Operations

`stack.push()`

`stack.pop()`

`stack.top()`

`stack.empty()`

`stack.size()`

`stack.push(5);`

`stack.push(4);`

`stack.push(3);`

`stack.pop();`

`stack.pop();`

5

Stack Operations

`stack.push()`

`stack.pop()`

`stack.top()`

`stack.empty()`

`stack.size()`

Return Value: Direct reference to the top element of the stack container.

```
cout << stack.top();
```

12
2
3
4
5

Stack Operations

`stack.push()`

`stack.pop()`

`stack.top()`

`stack.empty()`

`stack.size()`

Return Value: Direct reference to the top element of the stack container.

```
cout << stack.top();
```

Output: 12

12
2
3
4
5

Stack Operations

`stack.push()`

`stack.pop()`

`stack.top()`

`stack.empty()`

`stack.size()`

Returns :

True, if stack is empty

False, Otherwise

Output: False

12

2

3

4

5

Stack Operations

`stack.push()`

`stack.pop()`

`stack.top()`

`stack.empty()`

`stack.size()`

Output: 5

12
2
3
4
5

Print all elements in a stack

Output: 12, 2, 3, 4, 5

How can we do this?

12
2
3
4
5

Print all elements in a stack

Output: 12, 2, 3, 4, 5

```
while (!stack.empty()) {  
    cout << stack.top() << " ";  
    stack.pop();  
}
```

12
2
3
4
5

Implementing Stack with Array

```
struct StackArray {  
    int arr[MAX_SIZE];  
    int top;  
    int size; // Size of the stack  
  
    StackArray() {  
        top = -1; // Initialize top index to -1 (empty stack)  
        size = 0; // Initialize size to 0  
    }  
};
```

Implementing Stack with Array

```
void push(StackArray& stack, int x) {  
    if (stack.top >= MAX_SIZE - 1) {  
        cout << "Stack Overflow. Cannot push more elements." << endl;  
        return;  
    }  
  
    stack.arr[++stack.top] = x; // Increment top and insert element  
  
    stack.size++; // Increment size  
  
    cout << x << " has been pushed onto the stack." << endl;  
}
```

Implementing Stack with Array

```
int pop(StackArray& stack) {  
    if (stack.top < 0) {  
        cout << "Stack Underflow. Cannot pop from an empty stack." << endl;  
        return -1;  
    }  
  
    int poppedValue = stack.arr[stack.top--];  
    // Decrement top and return popped element  
    stack.size--; // Decrement size  
  
    return poppedValue;  
}
```


Implementing Stack with Array

```
int topValue(const StackArray& stack) {  
    if (stack.top < 0) {  
        cout << "Stack is empty." << endl;  
        return -1;  
    }  
  
    return stack.arr[stack.top];  
}
```

```
bool isEmpty(const  
StackArray& stack) {  
    return (stack.top == -1);  
}
```

Implementing Stack with Array

```
void print(const StackArray& stack) {  
    if (stack.top < 0) {  
        cout << "Stack is empty." << endl;  
        return;  
    }  
  
    cout << "Stack: ";  
  
    for (int i = stack.top; i >= 0; i--) {  
        cout << stack.arr[i] << " ";  
    }  
  
    cout << endl;  
}
```

Implementing Stack with Linked List

```
struct Node {  
    int key;  
    Node* next;  
};  
Node* top = nullptr;
```

Implementing Stack with Linked List

```
struct Node {  
    int key;  
    Node* next;  
};  
  
Node* top = nullptr;
```

```
void push(int x) {  
  
    Node* newNode = new Node;  
    newNode->key = x;  
    newNode->next = top;  
    top = newNode;  
  
    cout << x << " has been pushed onto  
the stack." << endl;  
}
```

Implementing Stack with Linked List

```
int pop() {  
    if (top == nullptr) {  
        cout << "Stack Underflow. Cannot pop from an empty stack." << endl;  
        return -1;  
    }  
  
    int poppedValue = top->key;  
    Node* temp = top;  
    top = top->next;  
    delete temp;  
  
    return poppedValue;  
}
```

Implementing Stack with Linked List

```
int topValue() {  
    if (top == nullptr) {  
        cout << "Stack is empty." << endl;  
        return -1;  
    }  
    return top->key;  
}
```

```
bool isEmpty() {  
    return (top == nullptr);  
}
```

Implementing Stack with Linked List

```
int size() {  
    int count = 0;  
    Node* current = top;  
    while (current != nullptr) {  
        count++;  
        current = current->next;  
    }  
    return count;  
}
```

Implementing Stack with Linked List

```
void print() {  
    if (top == nullptr) {  
        cout << "Stack is empty." << endl;  
        return;  
    }  
    ...  
}
```

```
Node* current = top;  
cout << "Stack: ";  
  
while (current != nullptr) {  
    cout << current->key << " ";  
    current = current->next;  
}  
cout << endl;  
}
```


Queue

- A linear data structure that follows the First In, First Out (FIFO) principle.
- The first element added to the queue is the first one to be removed.
- Think of it like a line of people waiting to be served: the person who arrived first gets served first.

Queue Operations

- Enqueue: `queue.push()`
- Dequeue: `queue.pop()`
- `queue.front()`
- `queue.back()`
- `queue.size()`
- `queue.empty()`

Queue

```
queue<int> myQueue;  
myQueue.push(10);
```

A diagram of a queue, represented as a horizontal rectangle with a black border. Inside the rectangle, the number 10 is centered.

10

Queue

```
queue<int> myQueue;  
myQueue.push(10);  
myQueue.push(20);
```



Queue

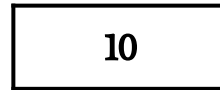
```
queue<int> myQueue;  
myQueue.push(10);  
myQueue.push(20);  
myQueue.push(30);
```

10	20	30
----	----	----

Queue

```
queue<int> myQueue;  
myQueue.push(10);  
myQueue.push(20);  
myQueue.push(30);  
myQueue.pop();
```

Popped Value = 10



Queue

`myQueue.front()`

Returns :

Direct reference to the first
element of the queue
container.

10	20	30
----	----	----

Output: 10

Queue

`myQueue.back()`

Returns :

Direct reference to the last
element of the queue
container.

10	20	30
----	----	----

Output: 30

Queue

```
myQueue.size()
```

Output: 3

10	20	30
----	----	----

Queue

`myQueue.empty()`

Returns **True** if the list is empty, else returns **False**.

10	20	30
----	----	----

Output: False

Implementing Queue Operations with Array

```
const int MAX_SIZE = 100; // Maximum size of the queue

struct Queue {
    int arr[MAX_SIZE];
    int front, rear;

    Queue() {
        front = -1; // Initialize front index to -1 (empty queue)
        rear = -1; // Initialize rear index to -1 (empty queue)
    }
};
```

Implementing Queue Operations with Array

```
void enqueue(Queue& q, int x) {  
    if (q.rear == MAX_SIZE - 1) {  
        cout << "Queue Overflow. Cannot enqueue more elements." << endl;  
        return;  
    }  
  
    if (q.front == -1) {  
        q.front = 0; // If queue is empty, set front to 0  
    }  
  
    q.arr[++q.rear] = x; // Increment rear and insert element  
    cout << x << " has been enqueued." << endl;  
}
```

Implementing Queue Operations with Array

```
int dequeue(Queue& q) {  
    if (q.front == -1 || q.front > q.rear) {  
        cout << "Queue Underflow. Cannot dequeue from an empty queue." << endl;  
        return -1;  
    }  
  
    int dequeuedValue = q.arr[q.front++]; // Increment front and return dequeued  
    element  
  
    return dequeuedValue;  
}
```

Implementing Queue Operations with Array

```
int front(const Queue& q) {  
    if (q.front == -1 || q.front > q.rear) {  
        cout << "Queue is empty." << endl;  
        return -1;  
    }  
  
    return q.arr[q.front];  
}
```

```
int back(const Queue& q) {  
    if (q.front == -1 || q.front > q.rear) {  
        cout << "Queue is empty." << endl;  
        return -1;  
    }  
  
    return q.arr[q.rear];  
}
```

Implementing Queue Operations with Array

```
bool empty(const Queue& q) {  
    return (q.front == -1 || q.front > q.rear);  
}
```

```
int size(const Queue& q) {  
    if (q.front == -1 || q.front > q.rear) {  
        return 0;  
    }  
    return q.rear - q.front + 1;  
}
```

Implementing Queue Operations with Array

```
void display(const Queue& q) {  
    if (empty(q)) {  
        cout << "Queue is empty." << endl;  
        return;  
    }  
  
    cout << "Queue: ";  
  
    for (int i = q.front; i <= q.rear; i++) {  
        cout << q.arr[i] << " ";  
    }  
  
    cout << endl;  
}
```

Implementing Queue Operations with Linked List

```
struct Node {  
    int data;  
    Node* next;  
};  
  
Node* front = nullptr;  
Node* rear = nullptr;
```


Implementing Queue Operations with Linked List

```
struct Node {  
    int data;  
    Node* next;  
};  
  
Node* front = nullptr;  
Node* rear = nullptr;
```

```
void enqueue(int x) {  
    Node* newNode = new Node;  
    newNode->data = x;  
    newNode->next = nullptr;  
  
    if (rear == nullptr) {  
        front = rear = newNode;  
    } else {  
        rear->next = newNode;  
        rear = newNode;  
    }  
}
```

Implementing Queue Operations with Linked List

```
int dequeue() {  
    if (front == nullptr) {  
        cout << "Queue is empty. Cannot  
        dequeue." << endl;  
        return -1;  
    }  
    ...  
}
```

```
int dequeuedValue = front->data;  
Node* temp = front;  
  
if (front == rear) {  
    front = rear = nullptr;  
} else {  
    front = front->next;  
}  
  
delete temp;  
return dequeuedValue;  
}
```

Implementing Queue Operations with Linked List

```
int frontValue() {  
    if (front == nullptr) {  
        cout << "Queue is empty." <<  
endl;  
        return -1;  
    }  
    return front->data;  
}
```

```
int rearValue() {  
    if (rear == nullptr) {  
        cout << "Queue is empty." <<  
endl;  
        return -1;  
    }  
    return rear->data;  
}
```

Implementing Queue Operations with Linked List

```
bool isEmpty() {  
    return (front == nullptr);  
}
```

```
int size() {  
    int count = 0;  
    Node* current = front;  
    while (current != nullptr) {  
        count++;  
        current = current->next;  
    }  
    return count;  
}
```

Implementing Queue Operations with Linked List

```
void display() {  
    if (front == nullptr) {  
        cout << "Queue is empty." << endl;  
        return;  
    }  
    ...  
}
```

```
    cout << "Queue (front to back): ";  
    Node* current = front;  
    while (current != nullptr) {  
        cout << current->data << " ";  
        current = current->next;  
    }  
    cout << endl;  
}
```